

Changelog

All notable changes to **vibechecker** are documented here. Format follows [Keep a Changelog](#).

[Unreleased] — feature/picoscope

Added

- **vibechecker/picoscope.py** — PicoScope 4000A acquisition backend (Phase 1)
 - `FindPicoScope()` — enumerates connected PS4000A units; returns `VibeSensor`-compatible dicts with `unit=['mV']`, mirroring the `FindDigiducer` interface
 - `PicoScopeStream` — background polling thread wrapping `ps4000aRunStreaming`
 - Converts ADC counts → mV via `adc2mV` on every driver callback
 - Accumulates variable-sized chunks into exact `blocksize` blocks before firing `DataCollector.recieve_data`
 - Implements `.active` / `.start()` / `.stop()` / `.close()` interface (compatible with `sounddevice.InputStream` and `SimulatedSensor`)
 - Handles USB-only / non-USB3 power states (status codes 282 / 286)
 - Reads back actual achieved sample rate after `ps4000aRunStreaming` and updates `AcquisitionSettings.samplerate`
- **AcquisitionSettings** — two new PicoScope-specific fields (`sample.py`):
 - `voltage_range: int = 8` — PS4000A range index (8 = PS4000A_5V)
 - `coupling: str = 'AC'` — channel A input coupling ('AC' or 'DC')
- **util.py** — extended `SAMPLERATES` list to include PicoScope-relevant rates: 100 kHz, 200 kHz, 500 kHz, 1 MHz
- **util.py** — added 'mV' to `SUPPORTED_UNITS` / `UNITS` dict for raw voltage passthrough
- **examples/ps4000a_triangle_stream_plot.py** — standalone script: generates a 500 Hz triangle wave (0.5 V amplitude, +1.4 V DC offset) via the PS4000A signal generator, streams Channel A at 50 kHz for 100 ms, then renders a Plotly HTML report with Welch PSD (10 windows, 50 % overlap) and top-5 peak detection

Changed

- **sensor.py** — `VibeSensor.find()` now calls `FindPicoScope()` instead of `FindDigiducer()`; `VibeSensor.connect()` returns a `PicoScopeStream` for hardware sensors and a `SimulatedSensor` for the simulation path
- **sensor.py** — removed `sounddevice` import and the `sounddevice` reset workaround; renamed internal `_callback` → `_sd_callback` (simulation path only)
- **collector.py** — removed `sounddevice` import; broadened `PortAudioError` catch in `start_stream()` to `Exception`; rewrote `recieve_data()` channel extraction to handle both (N, channels) 2-D arrays and 1-D arrays, with channel index clamping
- **sample.py** — `VibeSample.get_accel()` wraps `convert_units` in a try/except so unsupported conversions (e.g. 'mV' → 'g' before sensitivity is applied) pass through raw data instead of raising
- **README.md** — added PicoScope Integration section: architecture change, new `AcquisitionSettings` fields, Phase 1 data flow diagram, Phase 2 roadmap

Removed

- `digiducer.py` / `sounddevice` no longer used in the main acquisition path (file retained for reference; `FindDigiducer` still exported from `__init__.py`)
-

[0.1.0] — main (2025-xx-xx)

Initial public snapshot of the **sounddevice** / **Digiducer** acquisition path with:

- `VibeSensor` / `SimulatedSensor` / `DataCollector` pipeline
- `VibeSample` with Welch FFT, velocity spectrum, HDF5 save/load
- `AcquisitionSettings` with enforced interdependencies
- `dearpygui` GUI with real-time time-domain and frequency-domain plots
- Butterworth highpass filter (4th-order SOS, default 10 Hz cutoff)
- Simulated bearing-defect signals (`GenerateBearingVibration_SpectralMethod` , `GenerateBearingVibration_TemporalMethod`)
- Comprehensive README and pytest suite